Frontend Engineering: Day 2

JS Introduction, Variables, Declaration, Hoisting

Sayan Chowdhury
Engineer- Data Analytics
Design Automation

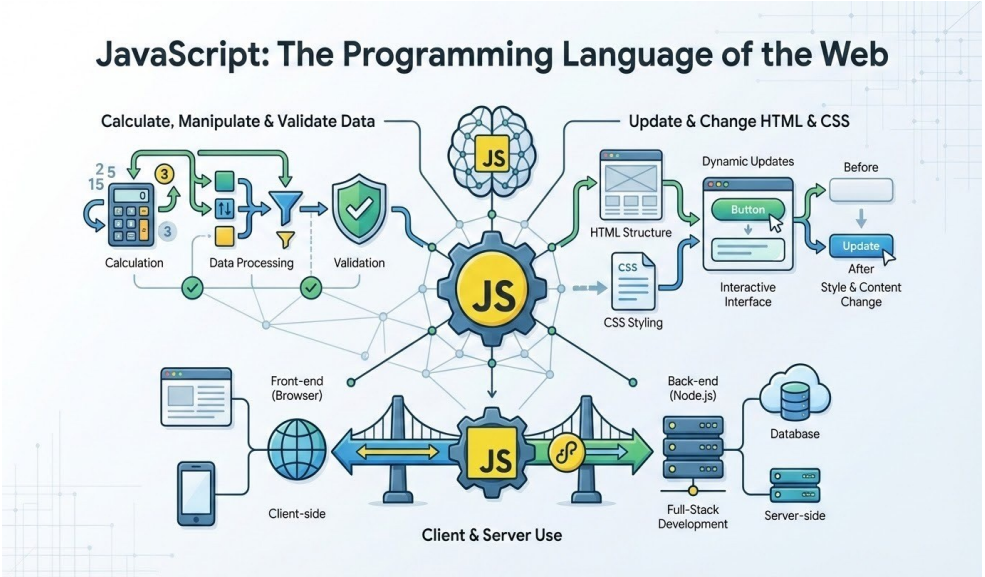
What is JavaScript?

JavaScript is the programming language of the web.

It can calculate, manipulate and validate data.

It can update and change both HTML and CSS.

It can be used in both client and server



JavaScript Where To?

The <script> Tag
In HTML, JavaScript code is inserted between *<script>* and *</script>* tags.

JavaScript in <head> or <body>
You can place any number of scripts in an HTML document.
Scripts can be placed in the *<body>*, or in the *<head>* section of an HTML page, or in both.

```
// Example
<script>
  document.getElementById("demo").innerHTML = "My First JavaScript";
</script>
```

JavaScript display output

JavaScript can display output in 4 main ways

1. Write into HTML Element:
innerHTML → renders HTML tags
innerText → shows plain text

```
document.getElementById("demo").innerHTML = "Hello";
document.getElementById("demo").innerText = "Hello";
```

2. Write to Browser Console
Used for debugging
Does not affect webpage UI

```
console.log("Hello");
console.table([1,2,3]);
```

3. Alert Popup

```
alert("I am");
window.alert("Happy");

```

4. Write to Page Directly

```
document.write("Hello")
//Only for testing
//Deletes page if used after load
```

Syntax Rules

The JavaScript syntax defines two types of values:

Literals (Fixed values)

Variables (Variable values)

```
// How to Declare variables:
let x = 5;
let y = 6;
// How to Compute values:
let z = x + y
// I am a Comment. I do Nothing
```

Syntax Rules

The most important syntax rules for literals (fixed values) are:

Numbers are written with or without decimals

Strings are text, written within double or single quotes:

JavaScript Keywords

JavaScript keywords are used to defines actions to be performed.
The *let* and *const* keywords create variables

JavaScript keywords are case-sensitive.
JavaScript does not interpret LET or Let as the keyword let.

```
// Example
let x = 55
const fname = "John";
```

JavaScript Statements

JavaScript statements are composed of:
Values, Operators, Expressions, Keywords, and Comments.

Semicolons separate JavaScript statements.
Add a semicolon at the end of each executable statement

```
let x, y, z;     // Statement 1
x = 5;           // Statement 2
y = 6;           // Statement 3
z = x + y;       // Statement 4
```

"JavaScript is a forgiving language. So if you don't use semicolons errors won't come in certain cases and it will run without error"

JavaScript Line Breaks & Comments

If a JavaScript statement does not fit on one line, the best place to break it is after an operator:

```
document.getElementById("demo").innerHTML =
  "Hello Dolly!";
```

Single line comments start with //

Any text between // and the end of the line will be ignored by JavaScript (will not be executed).

```
// Changeheading:
document.getElementById("myH").innerHTML =
"My First Page";

```

Multi-line comments start with /* and end with */.

Any text between /* and */ will be ignored by JavaScript.

```
/*The code below will change the heading with id = "myH"
and the paragraph with id = "myP" in my web page:
*/
document.getElementById("myH").innerHTML = "My First
Page";
```

JavaScript Variables

Variables = Data Containers

JavaScript variables are containers for data.

JavaScript variables can be declared in 4 ways

Modern JavaScript (ES6 – 2015)

Using *let : After the declaration, the variable has no value*

Using *const: Always use const if the value should not be changed*

Older JavaScript

Using *var*: var was used in all JavaScript code before 2015.

Automatically (Not Recommended)

```
let x = 5;
let y = 6;
let z = x + y;
/* The variable x is declared with the
let keyword.
The value of x can be changed. */
const x = 5;
const y = 6;
const z = x + y;
/* The two variables x and y are declared with
the const keyword.
They cannot be changed.*/
var x = 5;
var y
y = 6;

x=5
```

JavaScript Dollar Sign $

JavaScript also treats a dollar sign as a letter.

JavaScript Underscore (_)

JavaScript treats underscore as a letter.

Identifiers containing _ are valid variable names

JavaScript Variables

When to Use var, let, or const?

1. Always use const if the value should not be changed
2. Always use const if the type should not be changed (Arrays and Objects)
3. Only use let if you cannot use const
4. Never use var if you can use let or const.

One Statement, Many Variables

You can declare many variables in one statement.

Start the statement with let or const and separate the variables by comma

```
let person = "John Doe", carName = "Volvo", price = 200;
```

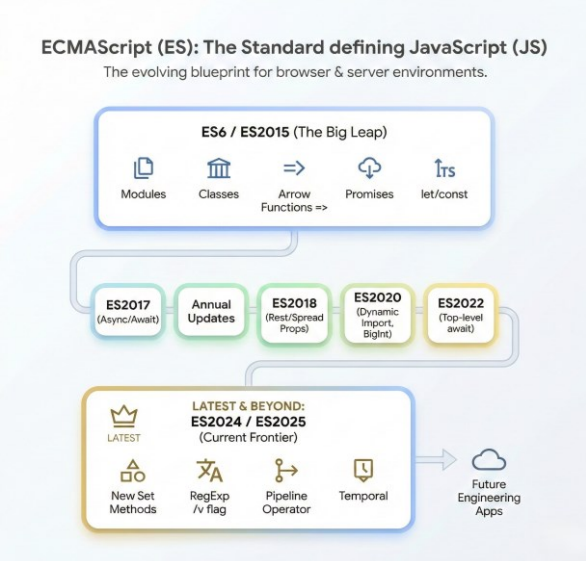
JavaScript Variables : Scope

Block Scope

- Before ES6 (2015), JavaScript did not have Block Scope.
- JavaScript had Global Scope and Function Scope.
- ES6 introduced the two new JavaScript keywords: let and const.
- These two keywords provided Block Scope in JavaScript

let:

- Variables declared with ***let*** have Block Scope
- Variables declared with ***let*** must be Declared before use
- Variables declared with ***let*** cannot be Redeclared in the same scope




```
Variables declared inside a { } block cannot be accessed from outside the block:
{
  let x = 2;
}
// x can NOT be used here
```

JavaScript Variables : Scope

Function Scope

Inside a function all variables declared with var, let or const have Function Scope

```
function myfunction() {
  var x = 1;
  let y = 2;
  const z = 3;
}
//x can NOT be used here
//y can NOT be used here
//z can NOT be used here
```

Global Scope

- Variables declared with the ***var*** always have Global Scope.
- Variables declared with the ***var*** keyword can NOT have block scope
- Variables declared with ***var*** inside a { } block can be accessed from outside the block

```
{
  var x = 2;
}
// x CAN be used here
```

JavaScript Variables : Redeclare

Variables defined with ***let*** can not be redeclared.

You can not accidentally redeclare a variable declared with ***let***.

With let you can not do this:

```
let x = "L&T";
let x = 0;
```

Variables defined with ***var*** can be redeclared.

With ***var*** you can do this:

```
var x = "L&T";
var x = 0;
```

IMPORTANT!

Redeclaring a variable using

the ***var*** keyword can impose problems.

Redeclaring a variable inside a block will also

redeclare the variable outside the block

```
var x = 10;
// Here x is 10
{
  var x = 2;
  // Here x is 2
}
// Here x is 2
```

JavaScript Variables : Redeclare

Redeclaring a variable using the ***let*** keyword can solve this problem.

Redeclaring a variable inside a block will not redeclare the variable outside the block

```
let x = 10;// Here x is 10
{
  let x = 2;// Here x is 2
}
// Here x is 10
```

With ***let***, redeclaring a variable in the same block is NOT allowed:

```
var x = 2;   // Allowed
let x = 3;   // Not allowed
{
  let x = 2;   // Allowed
  let x = 3;   // Not allowed
}
{
  let x = 2;   // Allowed
  var x = 3;   // Not allowed
}
```

Keyword	Scope	Redeclare
var	✓	✓
let	✓	✗
const	✓	✗

What will be the output of this code?

```
x = 5; // Assign 5 to x
console.log(x)
var x; // Declare x
```

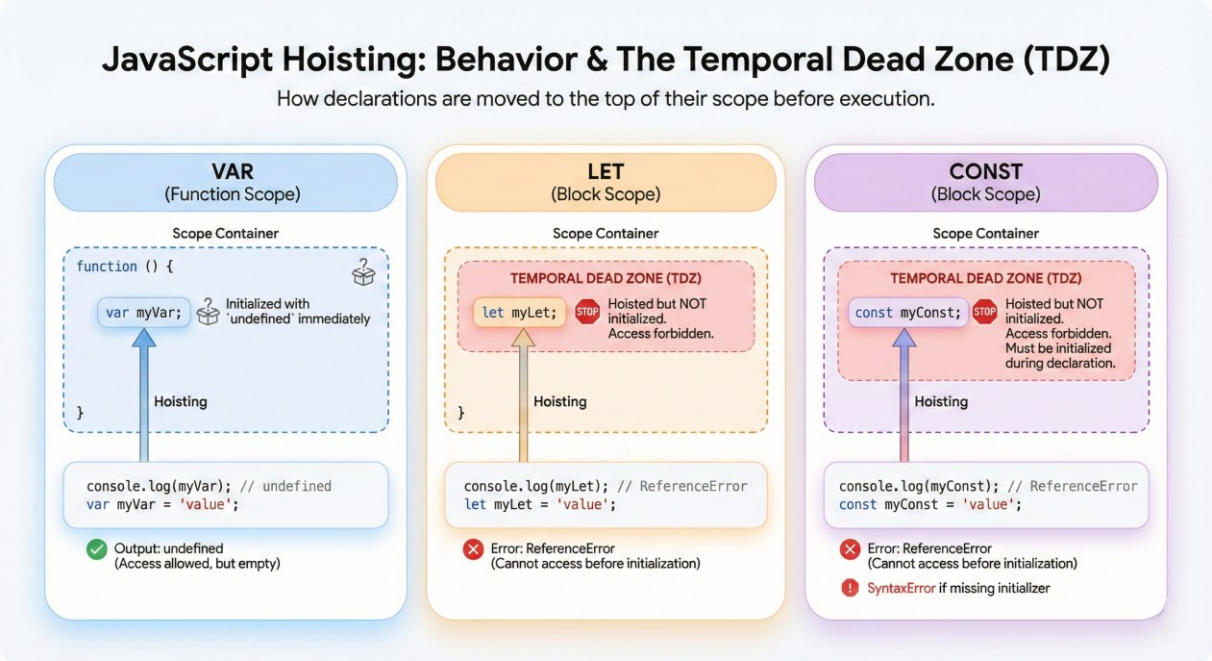
JavaScript Hoisting

JavaScript Declarations are Hoisted

In JavaScript, a variable can be declared after it has been used.

In other words; a variable can be used before it has been declared.

```
x = 5; // Assign 5 to x
console.log(x)
var x; // Declare x
```



JavaScript Hoisting

The let and const Keywords:

Variables defined with ***let*** and ***const*** are hoisted to the top of the block, but not initialized.

Meaning: The block of code is aware of the variable, but it cannot be used until it has been declared.

Using a let variable before it is declared will result in a **ReferenceError**.

The variable is in a "***temporal dead zone***" from the start of the block until it is declared:

```
carName = "Volvo";
let carName;
// Reference Error
```

```
carName = "Volvo";
const carName;
// Refrence Error
```

Thank **You**